COMSATS UNIVERSITY ISLAMABAD

Islamabad Campus

MACHINE LEARNING LAB — MIDTERM PROJECT REPORT

ShopSense

Customer Spending and Value Prediction System

Python | scikit-learn | FastAPI | HTML, CSS & JavaScript

Submitted by	Hassan Shahid
Registration No.	FA24-BBD-065
Program	BS Business Data Analytics
Section	5C
Submitted to	Mr. M. Ahmed
Date	30 October 2026

1. Introduction and Business Problem

Companies spend a lot of money on marketing, but not every customer is worth the same. Sending a loyalty offer to every customer wastes money on people who will not spend much, while missing the customers who would. In this project I used customer data from a food delivery and retail company to answer two questions:

- **Regression –** how much will a customer spend over two years?
- **Classification –** is the customer High-Value (spends 400 or more) or Regular?

I trained eight machine learning models, compared them, and connected the trained models to a website through a FastAPI backend, so the marketing team can enter a customer's details and get an answer straight away.

2. Dataset

I used the Customer Personality Analysis dataset (marketing_campaign.csv), which comes from the iFood marketing team. It has 2,240 customers and 29 columns. I chose it because it is real business data with a clear use, it is in the required size range, and it contains real problems to clean, such as missing values and wrong entries.

Source (Kaggle): https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis

Download copy used (original iFood file on GitHub): https://raw.githubusercontent.com/nailson/ifood-data-business-analyst-test/master/ml_project1_data.csv

Column	Type	Meaning
age (from Year_Birth)	Number	Customer's age in 2014, when the data was collected
education	Text	Basic, 2n Cycle, Graduation, Master, PhD
marital_status	Text	Single, Married, Together, Divorced, Widow
income	Number	Yearly household income
kids / teens	Number (0–2)	Small children / teenagers at home
recency	Number (0–99)	Days since the last purchase
web_visits	Number (0–20)	Visits to the company website last month
total_spending (target)	Number	Money spent on wine, fruit, meat, fish, sweets and gold in 2 years

Classification label: a customer who spent 400 or more is labelled High-Value (1), otherwise Regular (0). The value 400 is close to the median spending (397), which gives two almost equal groups (1,013 High-Value and 1,017 Regular). Because it is a fixed business rule, no information from the test data is used to create the label.

3. Data Preprocessing

Step	What was done	Records
1	Loaded raw data	2,240

2	Removed repeated customers (same data entered again with a new ID)	−182
3	Removed customers with missing income	−24
4	Removed impossible values: ages 114, 115 and 121, and one income of 666,666	−4
5	Changed odd marital values ("Alone", "Absurd", "YOLO") to "Single"	2,030 left
6	Added the six spending columns into total_spending and turned year of birth into age	–
7	One-hot encoded education and marital status (get_dummies, drop_first=True)	14 inputs
8	80% / 20% train-test split (random_state = 42, stratified)	1,624 / 406
9	StandardScaler for Logistic Regression, KNN, ANN and SVM, fitted on training data only	–

Income is in the tens of thousands while the number of kids is 0 to 2, so models that use distances or weights need scaling. Without scaling, Logistic Regression did not even finish learning.

Exploratory findings

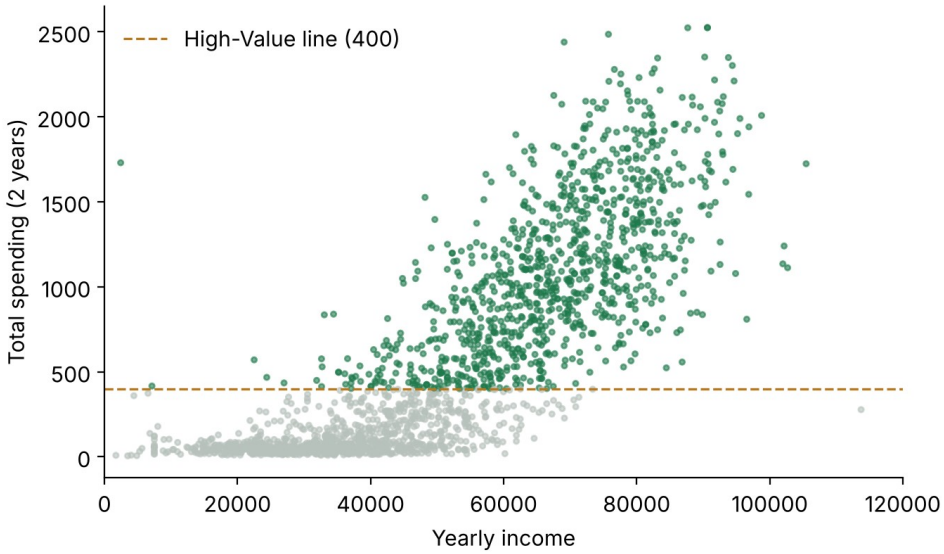


Figure 1: Income vs total spending. Green points are High-Value customers.

Income has the strongest link with spending (correlation 0.79). The relationship also bends upwards: spending grows much faster once income passes about 60,000. Customers with small children spend far less, and customers who visit the website more often also tend to spend less (correlation −0.49), probably because they are browsing for deals.

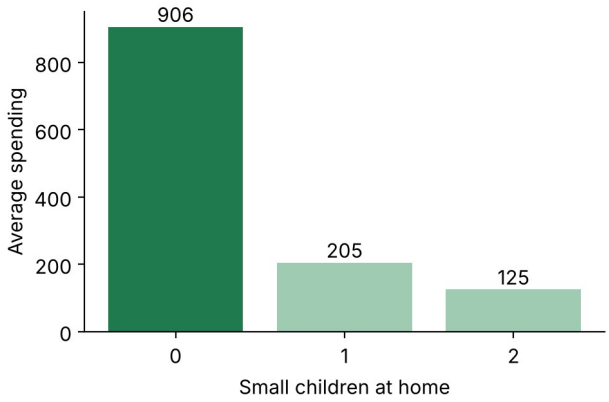


Figure 2: Average spending by number of small children.

4. Machine Learning Models

- **Linear Regression:** Fits a straight-line formula to predict total spending.
- **Logistic Regression:** Estimates the probability of being High-Value with the sigmoid curve; 50% or more means High-Value.
- **K-Nearest Neighbors (K = 15):** Takes a majority vote of the 15 most similar customers. K was chosen with 5-fold cross-validation.
- **Artificial Neural Network:** MLPClassifier with two hidden layers (16 and 8 neurons).
- **Support Vector Machine:** Finds the widest-margin boundary between the two groups (RBF kernel).
- **Decision Tree (max depth 4):** A set of yes/no rules. Depths 4 and 5 tied in cross-validation (89.41% vs 89.47%), so the simpler tree was kept.
- **Gaussian Naive Bayes:** Uses Bayes' theorem and assumes the columns are independent of each other.
- **Random Forest (100 trees):** Many decision trees vote together.

5. Results and Evaluation

5.1 Regression – Linear Regression

Metric	Value	Meaning
R^2 Score	0.7189	The model explains about 72% of the differences in spending
MAE	238.56	Average mistake in predicted spending
RMSE	303.52	Larger than MAE because some big spenders are badly predicted

About 71% of predictions are within 300 of the real spending. The model learned that each small child lowers spending by about 286 and each extra 1,000 of income adds about 21. However, 46 of the 406 test customers got a negative prediction. This happens because spending curves upwards with income, while Linear Regression can only draw a straight line. The website shows 0 in these cases.

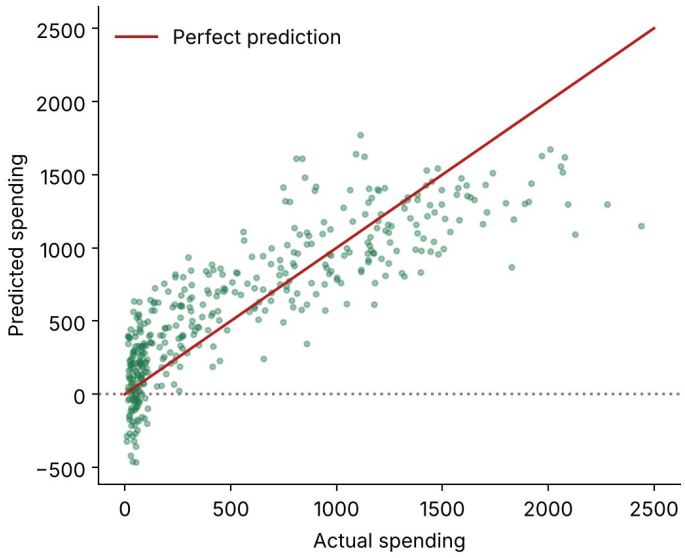


Figure 3: Actual vs predicted spending.

5.2 Classification – Seven Models

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	**90.64%**	**88.02%**	**94.09%**	**90.95%**
Decision Tree	89.66%	89.66%	89.66%	89.66%
ANN	89.41%	87.38%	92.12%	89.69%
Random Forest	89.16%	88.41%	90.15%	89.27%
SVM	89.16%	86.98%	92.12%	89.47%
Naive Bayes	87.19%	85.12%	90.15%	87.56%
KNN	85.71%	82.22%	91.13%	86.45%

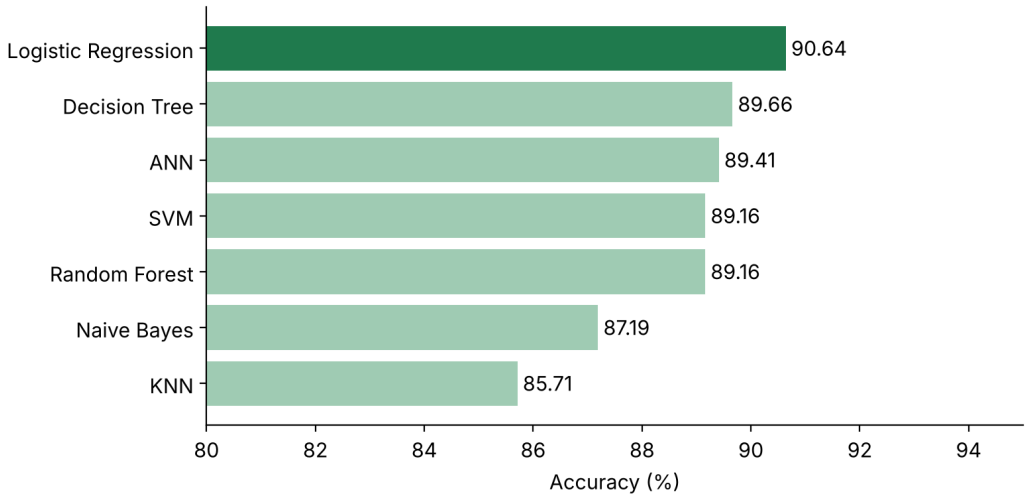


Figure 4: Test accuracy of each classifier on the same 406 customers.

5.3 Cross-Validation

One split of 406 customers could be lucky, so each classifier was also tested with 5-fold cross-validation on the training data. Scaling was placed inside a pipeline so it was re-learned in each fold.

Model	CV Mean Accuracy	Std. Deviation	Test Accuracy
Random Forest	89.90%	1.63	89.16%
Logistic Regression	89.65%	2.08	90.64%
Decision Tree	89.41%	1.62	89.66%
SVM	88.48%	1.50	89.16%
ANN	88.36%	1.94	89.41%
Naive Bayes	85.96%	1.38	87.19%
KNN	84.42%	0.98	85.71%

The top five models are within about 1.5% of each other, and the standard deviation is around 2%, so these five perform about equally well. Naive Bayes and KNN are clearly behind.

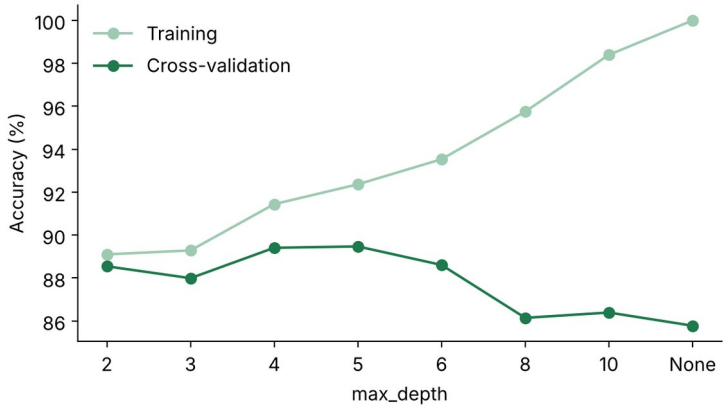


Figure 5: Training accuracy keeps rising with depth, but cross-validation accuracy peaks at depth 4–5.

5.4 Interpretation

- **Best model:** Logistic Regression had the best test accuracy (90.64%) and the highest recall (94.09%). It found 191 of the 203 High-Value customers in the test set, which matters because missing a good customer is a lost sale.
- **Simple is enough:** the simplest classifier did as well as the complex ones. Most of the decision comes from income and kids, and a straight boundary is enough to separate the groups.
- **Weakest models:** KNN treats all 14 columns as equally important when measuring distance, including education and marital columns that hardly matter. Naive Bayes assumes the columns are independent, but income, kids and web visits are related.
- **Overfitting:** an unlimited Decision Tree scored 100% on training data but only 85.8% in cross-validation, a clear case of memorising the data.
- **Where mistakes happen:** 34 of the 38 mistakes made by Logistic Regression were middle-income customers (34,000–68,000). 26 of them had a good income and usually no small children but still spent little, so on paper they looked like High-Value customers.

- **Key factors:** income is by far the most important feature (93% in the Decision Tree, 53% in the Random Forest), followed by small children and website visits.

6. FastAPI Backend

main.py loads all the saved .pkl files once when the server starts. Every request is checked with a Pydantic model: for example, age must be between 18 and 90 and education must be one of the five known values, otherwise the API returns an error. The customer is then one-hot encoded, matched to the same columns used in training, scaled if the model needs it, and passed to the model.

Method	Endpoint	Purpose
GET	/	Serves the website (index.html)
POST	/predict/regression	Expected spending in two years
POST	/predict/{model}	High-Value or Regular, with probability, for each of the 7 classifiers
GET	/model-performance	Comparison table of classifiers
GET	/regression-performance	R^2, MAE and RMSE
GET	/docs	Automatic interactive API documentation

I checked that the API gives exactly the same predictions as the models used directly in Python for 60 test customers and four models (240 checks, 0 differences).

7. Frontend and Integration

The frontend is one file, index.html. HTML builds the customer form, CSS handles the design and mobile layout, and JavaScript uses fetch() to send the customer's details to all 8 endpoints. The page shows the expected spending, each model's answer and confidence, a majority vote, and the model performance table loaded from the backend.

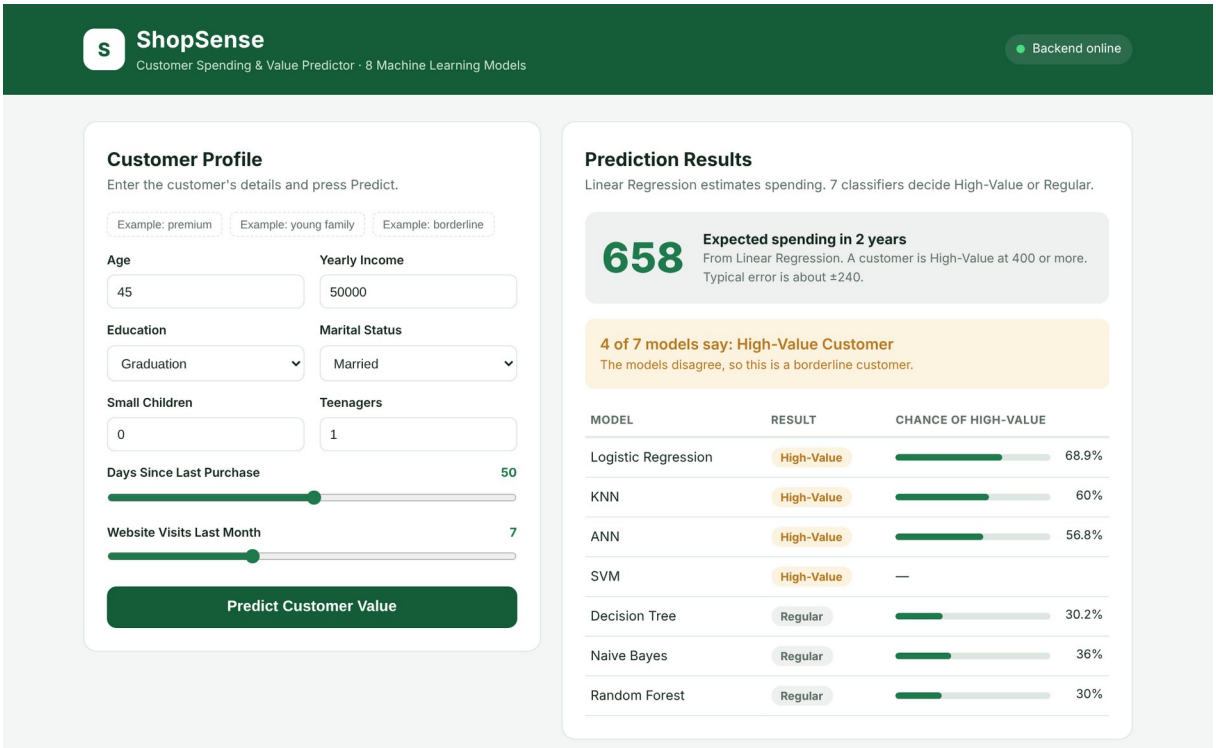


Figure 6: The ShopSense website showing a borderline customer, where the models split 4 to 3.

8. Setup Instructions

1. `pip install -r requirements.txt`
2. `python train_models.py` (cleans the data, trains and saves all 8 models)
3. `python evaluate_models.py` (creates the performance tables)
4. `uvicorn main:app --reload` (starts the backend)
5. Open `http://127.0.0.1:8000` in a browser

9. Limitations

The first limitation is the size of the data. After cleaning I had 2,030 customers, and only 406 of them were used for testing. With a test set this small, one or two customers can change the accuracy by about 0.5%, which is why the top five models cannot really be ranked against each other.

Second, Linear Regression is not a perfect fit for this problem. Spending grows faster at higher incomes, but the model can only draw a straight line, so it gave negative spending for some low-income customers. I set these to 0 on the website, but a model that can handle curves would probably predict spending better.

Third, the High-Value line of 400 is a rule I chose because it is close to the median. A real company might choose a different line depending on how much an offer costs, and the results would change with it.

Finally, I only used basic customer information. Some customers have a good income but still spend little, and nothing in these columns explains why. Information like past campaign responses or which shop a customer uses would probably help. The data is also from 2012–2014, so customer habits today may be different.

10. Conclusion

In this project I took a real marketing dataset through the full machine learning process. I found and fixed problems in the data, explored it with charts, trained and compared eight models, and built a working website on top of them using FastAPI.

For predicting spending, Linear Regression explained about 72% of the differences between customers, with an average error of about 239. For finding High-Value customers, Logistic Regression gave the best test result at 90.64% and found 94% of the real High-Value customers. Cross-validation showed that Logistic Regression, Random Forest, the Decision Tree, SVM and the ANN are all close, while KNN and Naive Bayes are weaker on this data.

The main thing I learned is that a more complex model is not automatically better. Logistic Regression is one of the simplest models I used, but it did as well as the neural network and is much easier to explain to a marketing team. I also saw overfitting clearly with the Decision Tree and learned to choose settings with cross-validation instead of the test set.

From a business point of view, income is the strongest sign of a valuable customer, while customers with small children and customers who visit the website often tend to spend less. The company could use the model to focus loyalty offers on the customers most likely to spend.